Transforming Relational Data into Graph Machine Learning Applications: Common Approaches and Examples

Rajiv Sambasivan

October 29, 2025

1 Overview and Scope

Graphs are a common analysis tool in data science and machine learning. In some cases, the need to use graphs is recognized upfront. The data model is designed to be a graph from the start. In other cases, the need to use graphs is recognized during the analysis phase. In such cases, the data is typically in a relational format. For example, data may come from an ERP (Enterprise Resource Planning)system, a CRM (Customer Relationship Management)system or Sales system (Salesforce). The challenge is to convert the relational data into a graph structure that is suitable for analysis. This document provides an overview of the concepts related to this transformation. The scope of the document is limited to the transformation of relational data into graph structures. Some familiarity with the relational model and the entity-relationship model is assumed. On the target side, the scope of this document is limited to the construction of homogeneous, bipartite and heterogeneous graphs. The document does not cover advanced topics such as hypergraphs, temporal graphs etc. The document stops at the point of graph construction. The document does not cover graph analysis or graph learning.

2 Motivations to Use Graphs

Typically when analysts want to explore graphs as a data representation, *heterogeneity* in the data is a key motivation. Heterogeneity is a term that originates in statistics. In statistics, heterogeneity refers to the presence of sub-populations within an overall population that exhibit different characteristics. For example, in a clinical trial, patients may respond differently to a treatment based on factors such as age. A company may have different customer segments that exhibit different purchasing behaviors. Heterogeneity is a common characteristic of business data. Graphs provide a natural way to represent and analyze

heterogeneous data. Graphs can capture the relationships between different entities in the data. Graphs can also capture the different types of entities in the data. This makes graphs a powerful tool for analyzing heterogeneous data.

How hetrogeneity manifests itself in data depends on the type of analysis being performed. There are three common types of data analysis: *cross-sectional, longitudinal* and *panel*. Each type of analysis has its own characteristics and challenges when it comes to identifying and accounting for heterogeneity in the data.

In a *cross-sectional* analysis, sources of heterogeneity may be identified through domain expertise or by applying data-driven techniques. A dataset where you are modeling housing prices over a wide geographic region at a specific point in time is an example of a *cross-sectional* dataset. A real estate domain expert may have developed ways to partition the housing market into segments where each segment has similar relationship between the house price and the characteristics of the hous. Alternatively, you can use machine learning to systematically uncover these sources. For example, you can use a Classification and Regression Tree (CART)[1] to identifiy regions that have a similar predictor-response characteristics and then develop models for each of these segments [29]. The regions identified by the regression tree are the sources of heterogeneity in the data.

In a *longitudinal analysis*, heterogeneity occurs over time. A study where you are monitoring the blood glucose of a subject over time, or, a study where you are tracking the price of a commodity like coffee over time is an example of a *longitudinal analysis*. Tracking changes with respect to time using *change point analysis* can tell you when and how many changes have occurred in the period that you are monitoring. See [27] for an example. Change point detection is a canonical time series analysis task and many methods such as the *Matrix Profile*[32] have been used to identify change points. Each segment between change points can be treated as a homogeneous segment. You can then model each segment separately. The segments identified by change point analysis are the sources of heterogeneity in the data.

In *Panel Data*, you need to account for sources of variation you have observed with both *cross-sectional* and *longitudinal* data. A company may track the purchasing behavior of a set of customers over time. In such cases, you need to account for heterogeneity across customers as well as heterogeneity over time.

3 Guidelines for Implementation

This section provides guidelines for implementing the transformation of relational data into graph structures. Definitions of key terms used in this section are provided in the Appendix.

The implementation consists of an initial assessment of the data, analysis for heterogeneity and a graph mapping. The guidelines for performing these steps are given below.

3.1 Initial Assessment

The initial assessment is done on the raw data obtained from the source systems. For the initial assessment, use a `jupyter notebook` to do the following:

1. Data Sourcing: Work with the data engineering team to understand how to get extracts from the source relational systems. Understand the data extraction process and the frequency of data extraction. Understand the data formats and the schema of the extracts. The data that you will source will be an estimate based on human judgement and understanding of what your use case aims to accomplish. This will be refined in subsequent analysis. Document the rationale for the estimate and the assumptions in a knowledge management system.

2. Data Profiling: Perform data profiling on the extracts to understand the data quality. Identify missing values, outliers, and inconsistencies in the data. Document your findings in a knowledge management system.

3. Data Selection: Based on your understanding of the use case, use an appropriate set of techniques, for example, feature selection techniques, to select the relevant data for your use case. Document your rationale for data selection in a knowledge management system.

4. Data Cleaning: Perform data cleaning to address the issues identified in the data profiling step. This may include handling missing values, removing outliers, and correcting inconsistencies in the data. Document your data cleaning process in a knowledge management system.

5. Data Transformation: Perform data transformation to convert the data into a format suitable for analysis. This may include normalization, aggregation, and encoding categorical variables. Document your data transformation process in a knowledge management system. Please see [10] and [11] for detailed guidelines on data transformation techniques.

6. Data Exploration: Perform exploratory data analysis to understand the characteristics of the data. This may include visualizations, summary statistics, and correlation analysis. The purpose of this step is to develop an initial understanding of the data and formulate a modeling plan. Document your findings in a knowledge management system.

7. Modeling Plan: Develop a preliminary modeling plan based on your understanding of the use case and the data. Document your modeling plan in a knowledge management system. The purpose of this document is to provide guidelies to prepare a graph model from relational data. The kind of graph model that you want to create is a key activity in this step. Please see the section 3.2 for guidelines related to performing this step.

8. Communication Plan: Identify the stakeholders for your use case and develop a communication plan to get validation and feedback on your

modeling results. Key performance indicators (KPIs) for the use case and how they relate to the model metrics should also be documented. Document your communication plan in a knowledge management system. This plan should include the frequency and format of communication with stakeholders.

A resource that is old but still relevant for a detailed discussion of the above steps is [16]. The book provides a good overview of data preparation tasks.

3.2 Graph Mapping

When you sit down to develop a modeling plan, you need to decide on the type of graph structure that is suitable for your use case. The choice of graph structure depends on the type of analysis you want to perform. This section discusses some common graph structures and their suitability for different types of analysis.

[6] and [7] provide a comprehensive discussion of some of the most common applications of graphs in enterprise applications. There are three common analysis themes relating to graph structure that we frequently encounter in enterprise applications. The first theme corresponds to a *homogeneous graph*. In such a graph, the nodes are the same type of entity. In these graphs there is a *pivotal entity* that is the focus of the analysis. For example, in a financial use case, the pivotal entity may be a *customer*. In a health care use case, the pivotal entity may be a *patient*.The analysis task typically involves predicting a property or an edge connecting these entities in the supervised learning scenario. Clustering of these entities is a common unsupervised learning scenario. If your use case is centered around a pivotal entity like a user, customer or patient, then this is a model you should consider.

The second frequently encountered theme in graph applications is the analysis of a *binary relationship*. In these applications the focus is on how two entities interact. THe analysis task is typically to extract patterns of such interactions. For example, in a retail use case, we may be interested in understanding the relationship between *customer* and *product*. In a health care use case, we may be interested in understanding the relationship between *patient* and *treatment*. In a manufacturing use case, we may be interested in understanding the relationship between *machine* and *operator*. In such cases, we may want to use a graph structure that is centered around the two entities of interest. This type of graph is a *bipartite* graph. See [12] for a detailed discussion of theory and applications of bipartite graphs. Personalization is also common motivation for this type of analysis. Summarizing the entity interactions so that you can develop personalization features is a common application task. *Machine learning for Personalization* is an emerging field, see [14] for a detailed discussion of applications and techniques. If your use case is centered around understanding the interaction between two entities, or developing personalization features, then this is a model you should consider.

Finally, the third common theme we counter is a small set of connected

entities and relationships. The analysis tasks in this case are typically understanding the behavior of or trying to predict:

1. The property of an specific entity (*node classification*)
2. The property of a relationship between two entities (*link prediction*)
3. The property of a collection of entities (*graph classification*)

For these problems, we can map the source entities and relationships to a graph structure that replicates the entity relationships in the source system. See [31] for the modeling approach and examples. In fact, learning on graphs with this setting is the focus of *Statistical Relational Learning*. [8] provides a detailed introduction to this area. The graphs corresponding to this analysis theme have different node types and an edge connects different node types. These type of graphs are called *heterogeneous graphs.Graph Neural Networks*, the use of neural networks to solve machine learning problems on graphs is another popular solution approach for the problems discussed in this sections, see [9] and [33] for details.

The analysis themes discussed above are by no means exhaustive. These are just the common themes that we have observed in enterprise use cases. Graphs have been used in many other ways to solve a variety of problems in engineering and science. For a discussion of other applications, for example to science and engineering, please see [4]. In many niche applications, such as fraud detection or de-duplication, the graph structure is evident from the problem statement itself. This document focuses on the common themes that we have observed in enterprise use cases.

3.3 Knowledge Graph Construction

As you work through the initial assement using the guidelines above, you will be making decisions based on the data that you have received. You may want to consider constructing a knowledge graph to capture this so that a maintainer of the model has a tool that inserts rationale at every decision point that explains the decision. *Knowledge Management for Data Science* (KMDS) [18] is python library that provides tools for constructing and maintaining knowledge graphs for data science projects. The ontology for the knowledge graph captures the activities that are described in section 3.1. The tool offers functionality for capturing the activities duiring the analysis and operationalization phases of the data science lifecycle. Section 4 provides an example of how to use KMDS to capture the graph construction process.

3.4 Modeling Result Review

When you have developed a modeling plan, reviewed the plan with stakeholders and have implemented the plan (selecting a graph model using the guidelines presented in section 3.2), you will need to review the results and interpret them.

The following two subsections provide guidelines for performing these tasks. Before you start these tasks, you will need to do the following:

1. Interpret the model metrics in the context of the use case KPIs.
2. Summarize the modeling guidelines for the current iteration. This would include level of feature engineering, model selection rationale, hyperparameter tuning approach and model validation approach.
3. Document the limitations of the current modeling approach.
4. Group results by stakeholder groups. Communicate implications of the results for each stakeholder group. A business stakeholder may be concerned with business impact and risks. A technical stakeholder may be concerned with model performance and operationalization. Make sure the right graphics and visualizations are used for each stakeholder group and reflect their concerns.
5. Develop a proposal for model assement for each stakeholder group.

3.5 Modeling Result Communication

Disseminate your modeling results to stakeholders using the communication plan developed during the initial assessment phase. Work through questions and feedback from stakeholders. Document the feedback in a knowledge management system.

3.6 Modeling Refinement Planning

Based on the feedback from stakeholders, develop a plan for refining the model. Based on the scope of the refinement, you may need buy in from project sponsors. Develop a refinement plan and develop the modeling plan for the next iteration. Document the refinement plan in a knowledge management system.

4 Examples of Implementation

4.1 Homogeneous Graphs

If your usecase pivots around one entity, its properties, or its relationships, then you should consider a *homogeneous graph* mapping. An example of using a *homogeneous* graph is provided with the data published by the U.S. Small Business Administration (SBA) on 7a loans. The data contains information about loans provided to small businesses by SBA approved lenders. The data contains information about the borrower, the loan amount, the purpose of the loan, the location of the business, and other relevant information. In this example, we develop models that explain the default behaviour of borrowers. There are three components to the analysis:

1. Develop a supervised learning model to predict loan default.
2. A key predictive feature is the proximity of a borrower to other borrowers that have defaulted. Similaity, or equivalently, the counterpart, dissimilarity is a key concept in this analysis. The key idea here is to construct a similarity be tween borrowers based on their characteristics. This similarity is then used to construct a graph where the nodes are borrowers and the edges represent similarity between borrowers. Please see [13] for a recent survey of similarity measures. A clustering of the subgraph representing the neighbors of a borrower is the second component of the analysis.
3. The analysis required to identify a threshold probablity of default that is used to classify borrowers as good or bad forms the third component of the analysis. Bad borrowers are close to other bad borrowers in the similarity graph. By knowing the properties of a bad borrower, we can use the model developed in the previous step to assign the credit worthiness of this borrower.

Please see [22] for details of developing a supervised learning model for predicting loan default. The details of the graph construction and the clustering analysis are provided in [26].

4.2 Bipartite Graphs

The second theme commonly encountered in developing a graph model is a *bipartite graph*. If your use case pivots around the analysis of a *pair* of entities and their relationships, you should consider the *bipartite* graph mapping. Bipartite graphs can be used to represent relationships between two types of entities. The example used for illustration is dervied from an example used to develop *descriptive analytics* on the shopping behavior of a retail store in Brazil. Please see [28] for of the analysis. The details of the bipartite graph construction and the subsequent analysis are provided in [17].

4.3 Heterogeneous Graph Construction from Relational Data

The third common theme we counter is a small set of connected entities and relationships. If your use case pivots around a *group* (typically, small, less than five) of entities, and thier relationships, you should consider the *heterogeneous* graph model. As part of the input to the analysis, we have the entities and the relationships between the entities. We treat these entities and relationships as a heterogeneous graph. The analysis tasks in this case are typically are:

1. Predict the property of an specific entity, *node classification*
2. Predict the relationship between two entities, *link prediction*
3. Predict the label associated with a new set of related entities, *graph classification*

A complete example of heterogeneous graph construction and analysis for node classification is provided in this article[30]. For guidelines and theoretical background for heterogeneous graph construction from relational data, please see [31]. For a comprehensive introduction to learning graphs from relational data, please see [8].

4.4 Knowledge Capture with KMDS

The decisions you have made during the initial assessment and graph mapping phases can be captured using KMDS. Please see [21] for details on how to use KMDS to capture the decisions you have made during the analysis. KMDS can be used to annotate your `jupyter notebooks` with the decisions you have made during the analysis. This will help you and your team to maintain the model over time. See [19] for an examples of how to use KMDS to annotate your machine learning and analytics workflows. KMDS was used to capture the process outlined in section 3.1 and section 3.2 for the SBA 7a loan dataset that was used as an example of homogeneous graph construction. A sketch of how the tool is used for initial assessment and graph mapping is provided here.

4.4.1 Data Tasks: Profiling, Cleaning, Transformation and Exploration

The data tasks outlined in section 3 can be captured using KMDS. The logging of the decision and rationale for each step is done using the `kmds` python package. Please see the following in the order given to understand how to use KMDS for logging data tasks:

1. Sourcing, Cleaning, Exploration and Profiling: Please see [20]
2. Data Transformation: Please see [25]
3. Data Transformation - Feature Engineering: Please see [24]

4.4.2 Model Planning

Please see section 4.1 for details of the modeling plan for the SBA 7a loan dataset. The modeling plan was captured using KMDS. Please see [23] for details of how the modeling plan was captured using KMDS.

4.4.3 Viewing the Knowledge Graph Sections

	obs_type	intent	finding	finding_seq
0	Data Quality Observation	DATA_UNDERSTANDING	Data Sourcing: The data was obtained from https://www.sba.gov/partners/lenders/7a-loan-program/types-7a-loans. The small business administration (SBA) makesit possible for small businesses to obtain working capital. They gaurantee borowers, lenders use this guaranteeto lend money to business owners. For more information, see this page. The SBA reports loan performance.Most loans are paid in full, a small fraction default. We can apply machine learning to determine which loans default.Please refer to the data dictionary in the data folder for detailed description of attributes in the dataset.	1
2	Data Quality Observation	DATA_UNDERSTANDING	Data Cleaning: Exclude small number of loans with unknown business implicationThere are about 3 loans which have the first disbursement date that is later than the date the loan was pain in full.This is either a data issue or, more likely, a business condition that is codified this way	2
1	Data Quality Observation	DATA_UNDERSTANDING	Data Cleaning: The Industry classification code, Zip Code, Borrower City and Bank Zip have category levels with less than 5 instances. For hierachical categories like ZipCode or NAICS code,maksing can be used to rollup these categories to the higher level. For flat structured categorical attributesthese instances were dropped.	3
3	Data Quality Observation	DATA_UNDERSTANDING	Data Cleaning: The notebook sba_loans.ipynb performs data cleaning and writes the intermediate filesba_loans_stage1.csv. This intermediate file is used to prepare the datasets for machine learning. This is the data that isencoded as a homogeneous graph	4
5	Data Quality Observation	DATA_UNDERSTANDING	Data Cleaning: The Industry classification code, Zip Code, Borrower City and Bank Zip have category levels with less than 5 instances. For hierachical categories like ZipCode or NAICS code,maksing can be used to rollup these categories to the higher level. For flat structured categorical attributesthese instances were dropped.	5
4	Data Quality Observation	DATA_UNDERSTANDING	Sequence of analysis notebooks, raw data to classifier: (1)sba_loans.ipynb (2) sba_loan_numeric_enc.ipynb(3) sba_bad_borr_graph_ctor.ipynb (4) sba_bad_loans_classifier.ipynb. For interpretation, see references provided in the knowledge base for this example.	6

(a) Data Activity Logging

	obs_type	intent	finding	finding_seq
0	Data Transformation Observation	DATA_UNDERSTANDING	data transformation: For the initial baseline, since the data dictionary is good and since,I have some familiarity with data in this domain, the atttribute set selected for modeling is based on reading the data dictionary. In a subsequent iteration, it makes sense to atleast confirm this with a mutual information based feature selection method. The dataset was also split into test and validation steps. A target encoding scheme was used to encode categorical features. All encoding was done using the training dataset (to avoid data leaks). The data loading is done by a utility class in the repository. It reads a configuration file: ../config/sba_loans/sba_homog_explicit.yml, to load the data.	1
1	Data Transformation Observation	DATA_UNDERSTANDING	data transformation: (feature engineering) A risky neighborhood feature was created using a kd-tree to identify the neighborhood of defaulted loans in the training set. If a loan belonged to this neighborhood, it has a value 1 otherwise it is 0. The bad loans were clustered using dbscan. For this iteration, two clusters were used. The distance of each loan in the training dataset to the cluster centroids is a new feature. Since we have two clusters, we have two new features.	2

(b) Data Transformation Logging

	obs_type	intent	finding
2	Modelling Choice Observation	MODEL_EXPLANATION	Modeling Plan: Sampling and thresholding are two of the most common approaches to developing a classifier for imbalanced data. The thresholding approach is used here. The probablity of a default is predicted by the model in this case a random forest model. A validation dataset is used to determine the threshold that gives us the desired trade off between precision and recall. Since we need high accuracy with probablity estimate, a probablity calibration model was also used. For details of the modeling, see https://github.com/rajivsam/descriptive_analytics/blob/main/examples/sba_7a_loans_analysis/sba_7a_loans_desc_analysis.pdf.
3	Modelling Choice Observation	MODEL_EXPLANATION	Modeling Plan: When the classifier tags a loan as bad, we need context to understand why it is bad. Thiscan be explained by recognizing that bad loans fall in the risky neighborbood (near other bad loans). By clustering therisky neighborhood and determining the cluster associated with a loan that is tagged as risky, you can get more contexton why this loan is determined to be bad by the mode. For details of the clustering and interpretation, see https://github.com/rajivsam/descriptive_analytics/blob/main/examples/sba_7a_loans_analysis/sba_7a_risky_borrower_analysis.pdf.
0	Modelling Choice Observation	MODEL_EXPLANATION	Modeling Review: (Put in your feedback from model review here.)
1	Modelling Choice Observation	MODEL_EXPLANATION	Modeling Refinement: (1)Refine Feature Engineering of Risky Neighborhood: Add more clusters, use a model selection method for clustering to add sufficient risky neighborhood clusters (2) Do feature selection usingmutual information to select the initial set of attributes.

(c) Model Planning and Refinement Logging

Figure 1: KMDS knowledge-graph sections captured during the SBA analysis

5 Use of Artificial Intelligence Assistants

AI assistants such as ChatGPT can be used to assist with the tasks outlined in this document. For example, AI assistants can be used to generate code snippets for data profiling, data cleaning, and data transformation. AI assistants can also be used to generate visualizations for exploratory data analysis. However, it is important to note that AI assistants are not a substitute for human expertise. The use of AI assistants should be complemented with human judgement and domain knowledge. The final decisions should always be made by a human analyst. For over two decades at the time of this writing, companies and individuals have believed that it is possible to point a tool at a datsource and expect a nice robust model to be produced by the tool. To paraphrase [16],

> *Some companies seem to have the impression that in order to produce effective models, knowledge of the data and the problem are not really required, but that the tools will do all the work. Where this myth came from is hard to imagine. It is so far from the truth that it would be funny if it were not for the fact that major projects have failed entirely due to ignorance on the part of the miner. Not that the miner was always at fault. If ordered to "find out what is in this data," an employee has little option but to do something. No one who expected to achieve anything useful would approach a lump of unknown substance, put on a blindfold, and whack at it with whatever tool happened to be at hand. Why this is thought possible with data mining tools is difficult to say!*

Building robust models requires human expertise and judgement. AI assistants can be a useful tool in the data science toolkit, but they should not be relied upon exclusively.

References

[1] Leo Breiman et al. *Classification and regression trees*. Chapman and Hall/CRC, 2017.

[2] Peter Pin-Shan Chen. "The entity-relationship model—toward a unified view of data". In: *ACM transactions on database systems (TODS)* 1.1 (1976), pp. 9–36.

[3] Edgar F Codd. "A relational model of data for large shared data banks". In: *Communications of the ACM* 13.6 (1970), pp. 377–387.

[4] Diane J Cook and Lawrence B Holder. *Mining graph data*. John Wiley & Sons, 2006.

[5] Ramez Elmasri and Shamkant B Navathe. *Fundamentals of database systems seventh edition*. pearson, 2016.

[6] al. Faloutsos Christos et. *User Behavior Modeling Part*. Aug. 3, 2025. URL: https://www.youtube.com/watch?v=fMHZt4UStUg&list=PL-lbroKJyNLCAM85ENZ8g2DEiG1e5TQEo&index=1.

[7] al. Faloutsos Christos et. *User Behavior Modeling Part*. Aug. 3, 2025. URL: https://www.youtube.com/watch?v=eI-hmySej_w&list=PL-lbroKJyNLCAM85ENZ8g2DEiG1e5TQEo&index=2.

[8] Lise Getoor and Ben Taskar. *Introduction to statistical relational learning*. MIT press, 2007.

[9] William L Hamilton. *Graph representation learning*. Morgan & Claypool Publishers, 2020.

[10] Max Kuhn, Kjell Johnson, et al. *Applied predictive modeling*. Vol. 26. Springer, 2013.

[11] Max Kuhn and Kjell Johnson. *Feature Engineering and Selection: A Practical Approach for Predictive Models — bookdown.org*. https://bookdown.org/max/FES/. [Accessed 25-10-2025].

[12] Jérôme Kunegis. "Exploiting the structure of bipartite graphs for algebraic and spectral graph theory applications". In: *Internet Mathematics* 11.3 (2015), pp. 201–321.

[13] Avivit Levy, B Riva Shalom, and Michal Chalamish. "A guide to similarity measures". In: *arXiv preprint arXiv:2408.07706* (2024).

[14] Julian McAuley. *Personalized machine learning*. Cambridge University Press, 2022.

[15] *ml-ops.org — ml-ops.org*. https://ml-ops.org/. [Accessed 25-10-2025].

[16] Dorian Pyle. *Data preparation for data mining*. morgan kaufmann, 1999.

[17] Rajiv Sambasivan. *CoClustering of Temporal Shopping Data (Olist Case Study)*. https://github.com/rajivsam/descriptive_analytics/blob/main/examples/olist_case_study/temporal_coclustering_SP_2017.pdf. [Accessed 28-10-2025].

[18] Rajiv Sambasivan. *GitHub - rajivsam/KMDS: Source for KMDS — github.com*. https://github.com/rajivsam/kmds. [Accessed 06-09-2025].

[19] Rajiv Sambasivan. *Home — github.com*. https://github.com/rajivsam/kmds_recipes/wiki. [Accessed 28-10-2025]. 2024.

[20] Rajiv Sambasivan. *Jupyter Notebook, SBA loans dataset- initial assement,– github.com*. https://github.com/rajivsam/descriptive_analytics/blob/main/notebooks/sba_loans.ipynb. [Accessed 28-10-2025].

[21] Rajiv Sambasivan. *KMDS/feature_documentation/km_app_pipeline.md at main · rajivsam/KMDS — github.com*. https://github.com/rajivsam/KMDS/blob/main/feature_documentation/km_app_pipeline.md. [Accessed 28-10-2025].

[22] Rajiv Sambasivan. *Predicting Bad Loans (SBA 7a dataset)*. https://github.com/rajivsam/descriptive_analytics/blob/main/examples/sba_7a_loans_analysis/sba_7a_loans_desc_analysis.pdf. [Accessed 28-10-2025].

[23] Rajiv Sambasivan. *SBA Bad Loans Classifier*. https://github.com/rajivsam/descriptive_analytics/blob/main/notebooks/sba_bad_loans_classifier.ipynb. [Accessed 28-10-2025].

[24] Rajiv Sambasivan. *SBA Dataset Risky Neighborhood Feature Engineering*. https://github.com/rajivsam/descriptive_analytics/blob/main/notebooks/sba_bad_borr_graph_ctor.ipynb. [Accessed 28-10-2025].

[25] Rajiv Sambasivan. *SBA dataset, Data Transformation- github.com*. https://github.com/rajivsam/descriptive_analytics/blob/main/notebooks/sba_loan_numeric_enc.ipynb. [Accessed 28-10-2025].

[26] Rajiv Sambasivan. *SBA risky borrower analysis*. https://github.com/rajivsam/descriptive_analytics/blob/main/examples/sba_7a_loans_analysis/sba_7a_risky_borrower_analysis.pdf. [Accessed 28-10-2025].

[27] Rajiv Sambasivan. *Understanding Coffee Prices*. https://rajivsam.github.io/r2ds-blog/posts/markov_analysis_coffee_prices/. Accessed: 2025-06-21. 2025.

[28] Rajiv Sambasivan. *What Descriptive Analytics Reveals About Shopping Behavior (Olist Case Study)*. https://github.com/rajivsam/descriptive_analytics/blob/main/examples/olist_case_study/picture_summary_SP_2017.md. [Accessed 28-10-2025].

[29] Rajiv Sambasivan and Sourish Das. "Classification and regression using augmented trees". In: *International Journal of Data Science and Analytics* 7.4 (2019), pp. 259–276.

[30] Jörg Schad, Rajiv Sambasivan, and Christopher Woodward. "Predicting help desk ticket reassignments with graph convolutional networks". In: *Machine Learning with Applications* 7 (2022), p. 100237. ISSN: 2666-8270. DOI: https://doi.org/10.1016/j.mlwa.2021.100237. URL: https://www.sciencedirect.com/science/article/pii/S2666827021001195.

[31] Michael Schlichtkrull et al. "Modeling relational data with graph convolutional networks". In: *European semantic web conference*. Springer. 2018, pp. 593–607.

[32] Chin-Chia Michael Yeh et al. "Matrix profile I: all pairs similarity joins for time series: a unifying view that includes motifs, discords and shapelets". In: *2016 IEEE 16th international conference on data mining (ICDM)*. Ieee. 2016, pp. 1317–1322.

[33] Jie Zhou et al. "Graph neural networks: A review of methods and applications". In: *AI open* 1 (2020), pp. 57–81.

Appendix: Vocabulary and Definitions

The Relational Model

[3] introduced the relational model, which is the logical foundation for relational databases. In this model, data is represented as relations (tables), where each relation consists of tuples (rows) and attributes (columns). An algebraic framework for manipulating these relations is provided by relational algebra, which includes binary operations (operate on two relations) such as join and union, as well as unary operations like selection and projection (operate on a single relation). The relational model provides a mathematical framework for representing and querying data.

The Entity-Relationship Model

The entity-relationship (ER) model, introduced by [2] is a framework for describing the *semantic* data abstractions and their relationship with each other. For example, if your use case is about how students in a university register for courses, then *students, course, registration* might be a set of data abstractions. The *registration* may connect a student with a course for a particular semester. From a *data representation* point of view, all semantic abstractions may be represented as relations.

The Entity-Relationship to Relational Schema Mapping

There exist guidelines and a body of knowledge around mapping an *Entity-Relationship Model* to a *Relational Schema*. See [5], for example. The working context for this document that you have a set of extracts from a relational database, in other words, this step has been done.

Use Case Data

Your *Use Case Data* is what you want to convert to a graph. The data for your use case is a set of extracts that correspond to the entities and their relationships. As a modeler, you are expected to know:

- The semantic abstractions relevant to your use case. These are the *entities* in your data extract. You should spend sometime to develop a clear understanding of the entities in your use case. If you don't have a data dictionary for your extracts, you should work with the data engineering team to develop one.
- The *semantic relationships* in your usecase. You should understand how *entities* in your usecase are related. You should develop clear definitions for these relationships. The impact of these relationships on your usecase should also be documented.

Model Operationalization

When analysis is complete, we have a plan for how data is going to be processed and fed as input to models. Operationalizing the model and monitoring the model is the next step. This is typically done by an MLOps team. For details on MLOps, please see [15]